

COS 433: Cryptography

Reminder:

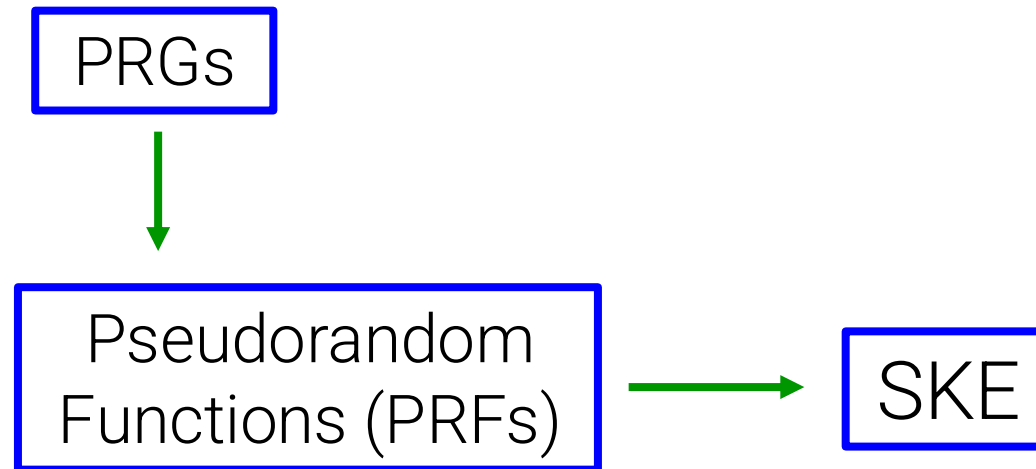
- Problem Set 1 due 9/23.
- Problem Set 2 will be released 9/23

“The PRG Conjecture”

Main Conjecture: PRGs with one bit of stretch exist

If true, we have secret-key cryptography!

Plan: Secret-key encryption from PRGs



Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Notation: If (Enc, Dec) is a secret-key encryption scheme with message length $n = n(\lambda)$ and $\mathbf{m} \in (\{0,1\}^n)^t$, we define $\text{Enc}(k, \mathbf{m}) = (\text{Enc}(k, m_1), \dots, \text{Enc}(k, m_t))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

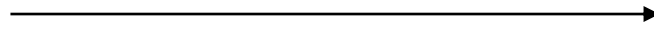
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: for $t \geq 2$, encryption can no longer be deterministic!

$$\text{Enc}(\textcolor{red}{k}, m_0), \text{Enc}(\textcolor{red}{k}, m_0) \approx_c \text{Enc}(\textcolor{red}{k}, m_0), \text{Enc}(\textcolor{red}{k}, m_1)$$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

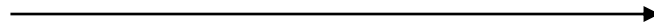
$$\text{For all } \mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right), \quad \boxed{\text{Enc}(\textcolor{red}{k}, \mathbf{m}_0) \approx_c \text{Enc}(\textcolor{red}{k}, \mathbf{m}_1)}.$$

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; \mathbf{r})$$



k

$$m' = \text{Dec}(k, ct)$$

Notation: If (Enc, Dec) is a secret-key encryption scheme with message length $n = n(\lambda)$ and $\mathbf{m} \in (\{0,1\}^n)^t$, we define $\text{Enc}(k, \mathbf{m}) = (\text{Enc}(k, m_1; \mathbf{r}_1), \dots, \text{Enc}(k, m_t; \mathbf{r}_t))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

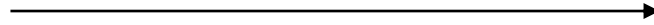
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; r)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: without loss of generality, we can take $n(\lambda) = 1$ (one-bit messages)!

Redefine $\text{Enc}(k, m \in \{0,1\}^n) = (\text{Enc}(k, m_1), \dots, \text{Enc}(k, m_n))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

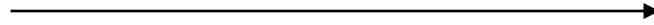
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(k, \mathbf{m}_0) \approx_c \text{Enc}(k, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; r)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: without loss of generality, we can take $n(\lambda) = 1$ (one-bit messages)!

Exercise: nt -message security (one-bit messages) $\rightarrow$ t -message security (n -bit messages)

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(k, \mathbf{m}_0) \approx_c \text{Enc}(k, \mathbf{m}_1)$.

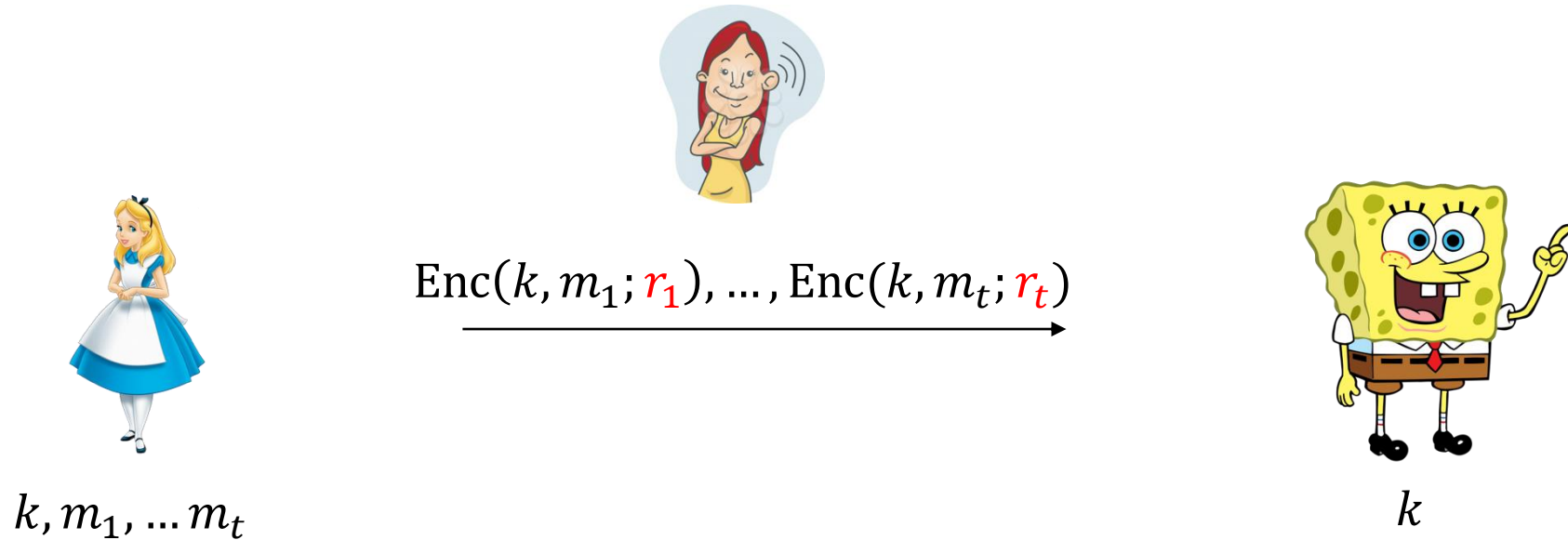
Secret-Key Encryption (multiple messages)

Is this what we want?

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

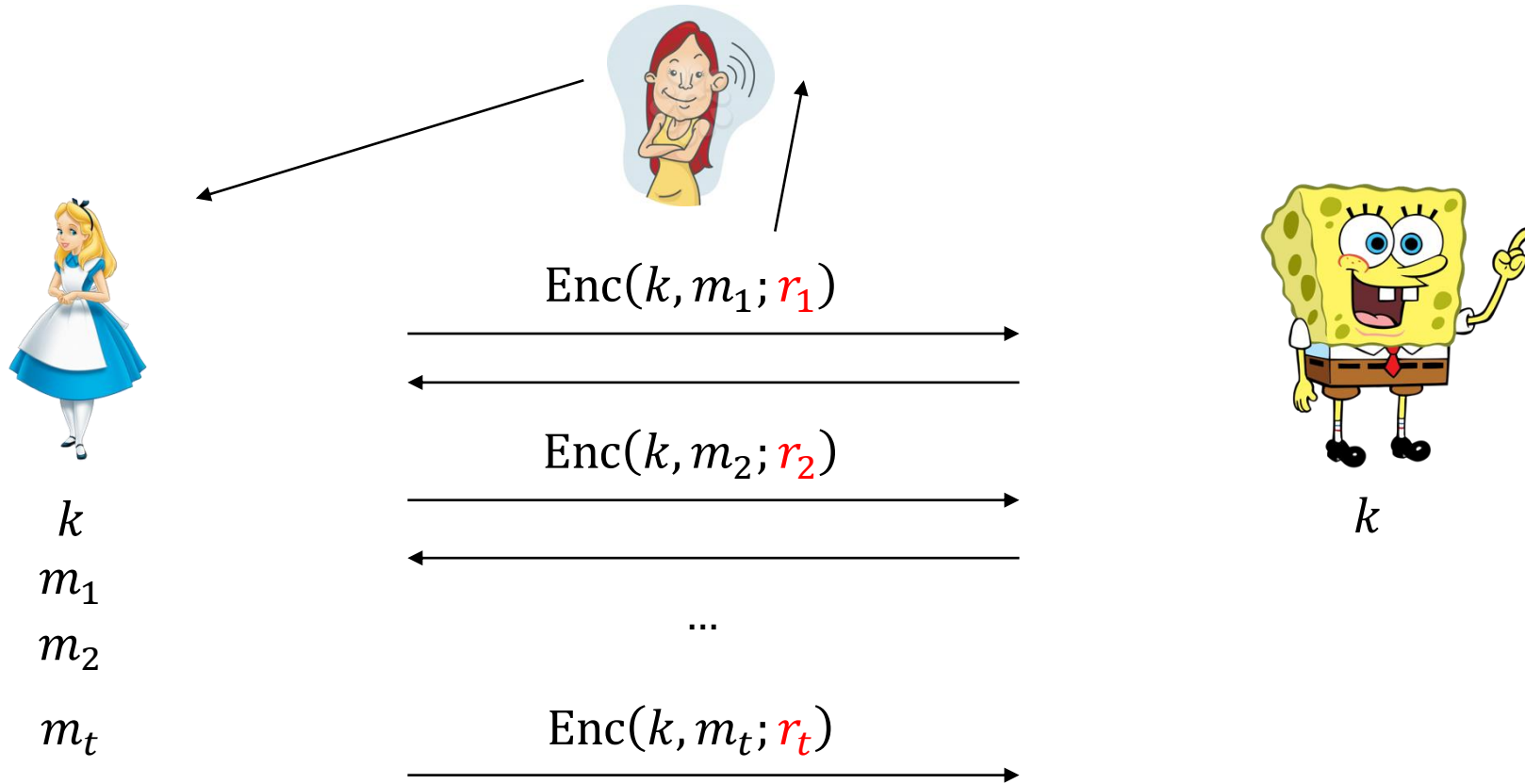
Secret-Key Encryption (multiple messages)



Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

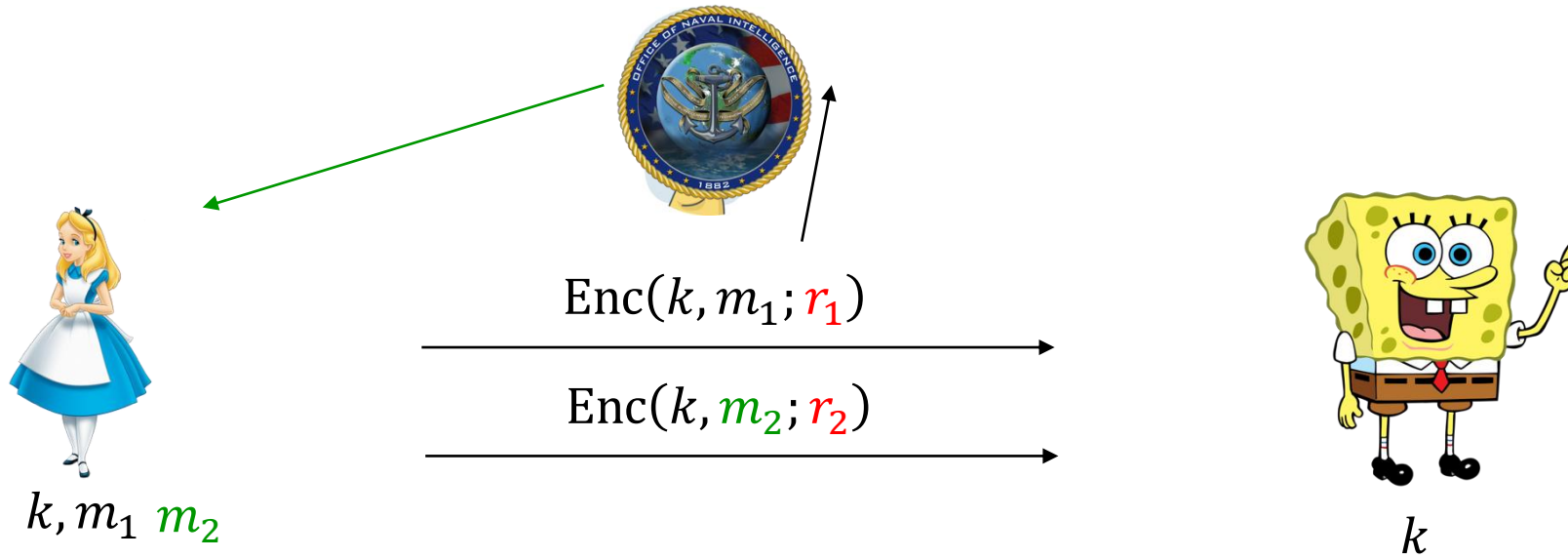
Secret-Key Encryption (multiple messages)



What information **can m_i depend on?**

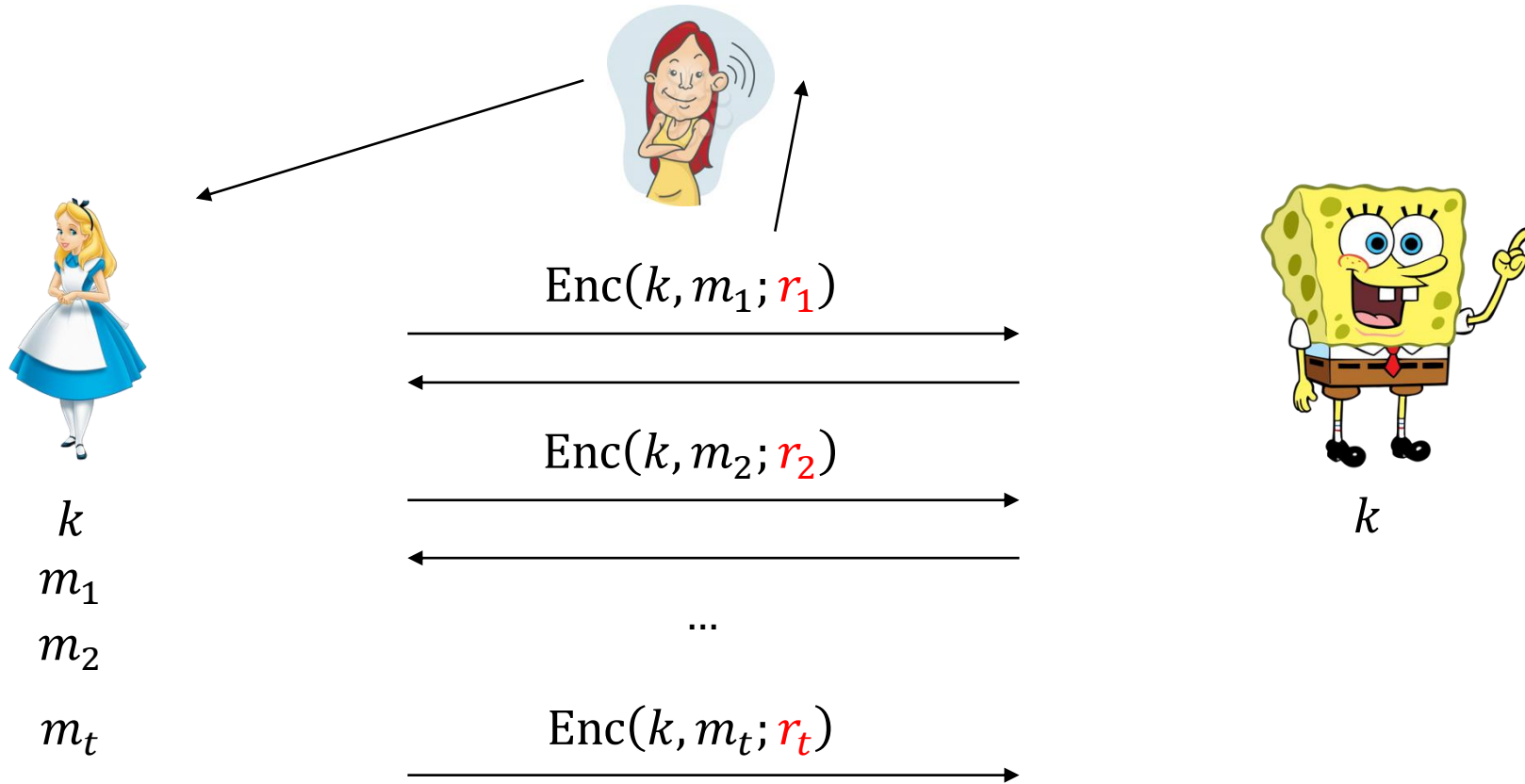
- Messages (encrypted or unencrypted) that Bob sends back to Alice
- The weather on day i
- Actions that the adversary  takes.

An “known plaintext attack”



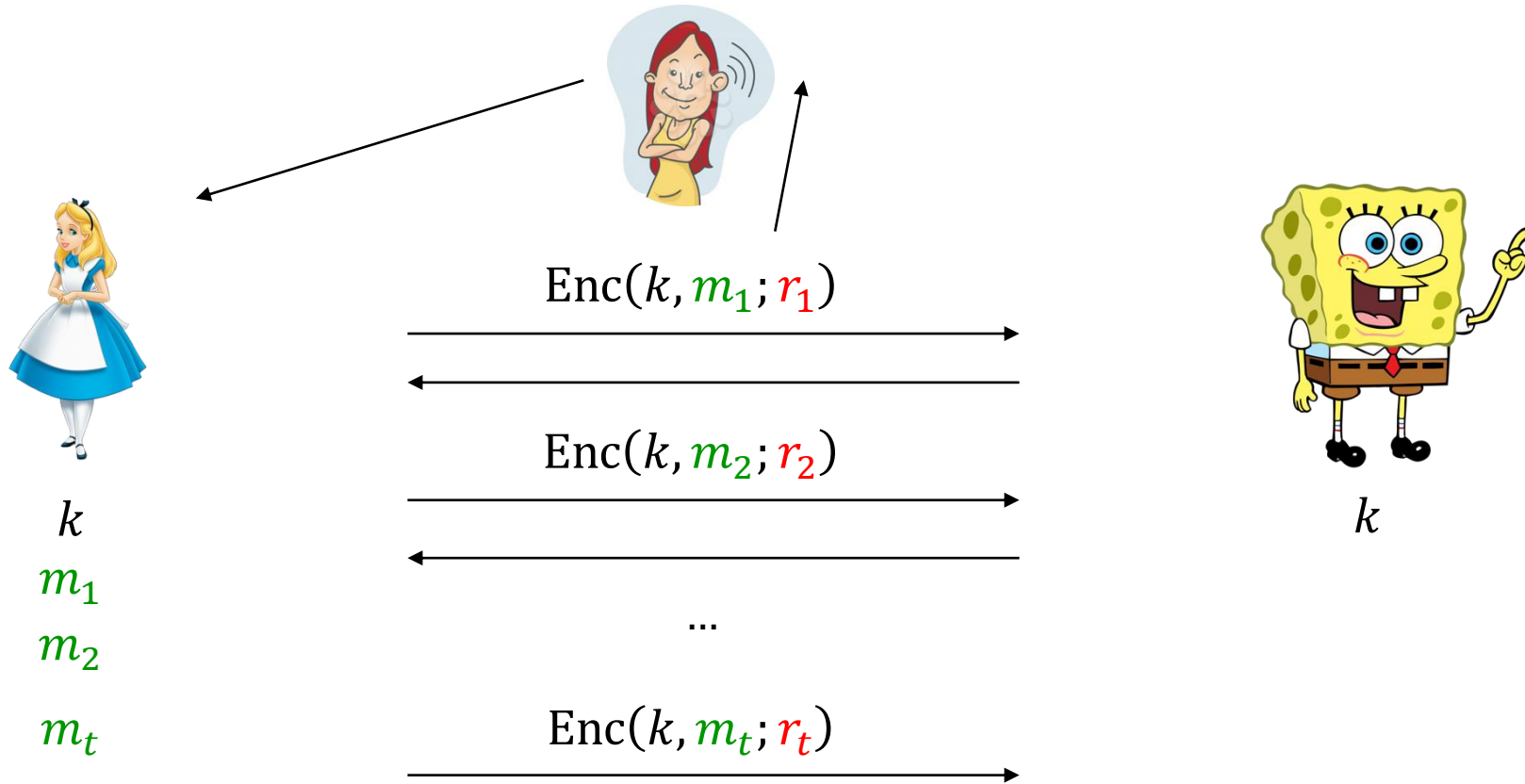
- WWII: US Navy eavesdrops that Japan plans to attack location “AF”
- Previous knowledge: locations near Hawaii have codewords starting with A
- Educated guess: “AF” refers to Midway island
- US releases a fake message about Midway island in the clear (“low supplies”)
- Japan sends reports about “AF” having low supplies.

Secret-Key Encryption (multiple messages)



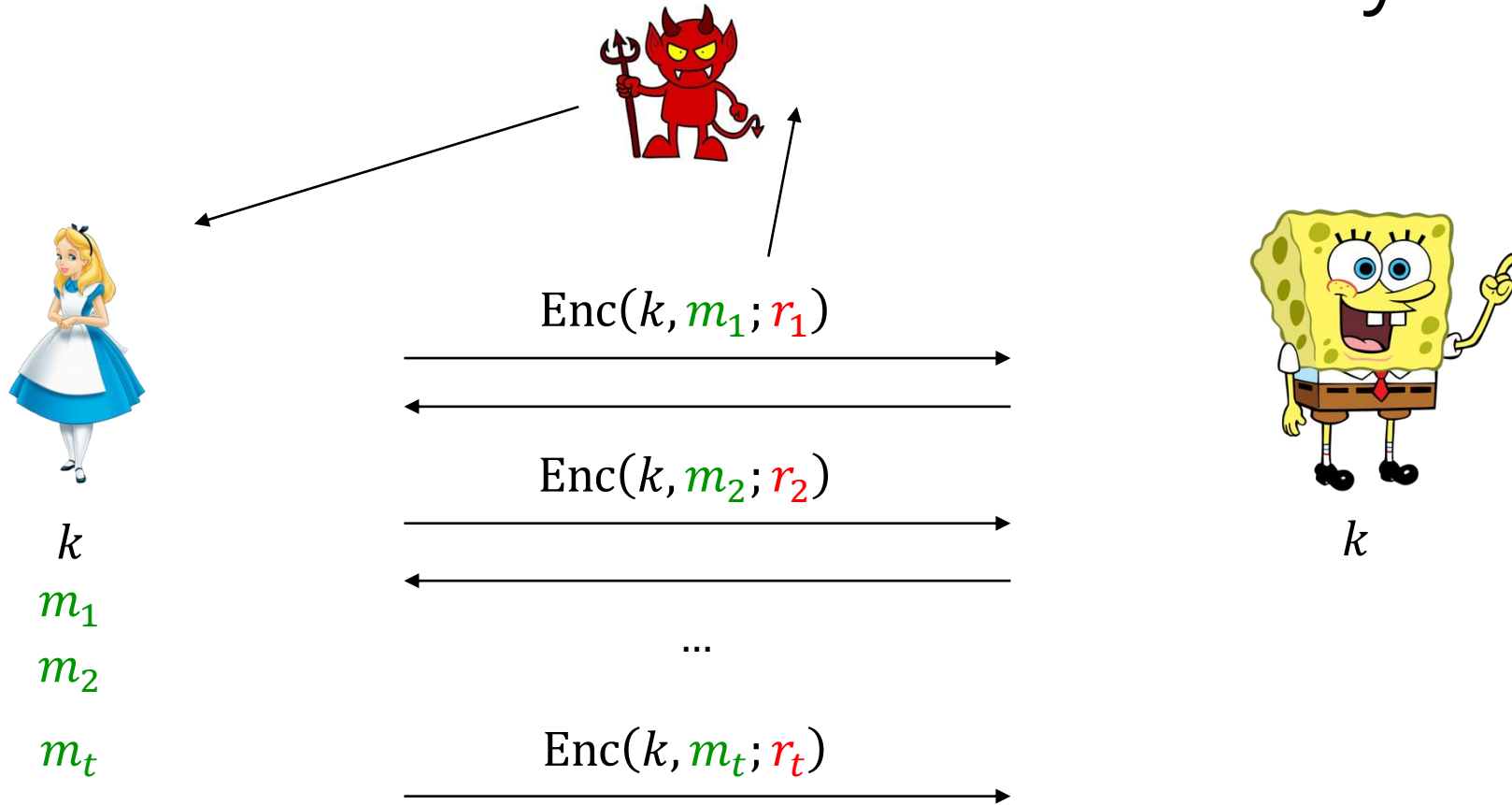
How do we model this dependency?

Chosen-Plaintext Security



Cautious model: adversary has **complete power** over the choice of messages.

Chosen-Plaintext Security



Cautious model: adversary has **complete power** over the choice of messages.

Game-based security definitions

IND-Security, Again

Challenger/Honest Party



k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$



$\text{Enc}(k, m_b; r)$



Adversary



b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $m_0 = \left(m_0^{(\lambda)} \in \left(\{0,1\}^{n(\lambda)}\right)^{t(\lambda)}\right)_\lambda, m_1 = \left(m_1^{(\lambda)}\right)$, $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$.

IND-Security, Again

Challenger/Honest Party



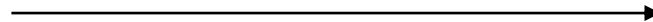
k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$



$\text{Enc}(k, m_b; r)$



Adversary



b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all polynomial-time



$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

IND-CPA Security

Challenger/Honest Party



Sampled once

k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\text{Enc}(k, m_b; r)$

New Enc randomness
each time

Adversary



b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-CPA) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all polynomial-time



$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

IND-CPA Security, Alt. Definition

Challenger/Honest Party



k

$b \leftarrow \{0,1\}$

$m^{(\lambda)}$

$\text{Enc}(k, m; r)$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

Challenge: $\text{Enc}(k, m_b; r)$

$m^{(\lambda)}$

$\text{Enc}(k, m; r)$

Adversary



b'

Win if $b' = b$

For all polynomial-time



$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

IND-CPA Security

Lemma: Definition and Alt. Definition are equivalent

Proof:

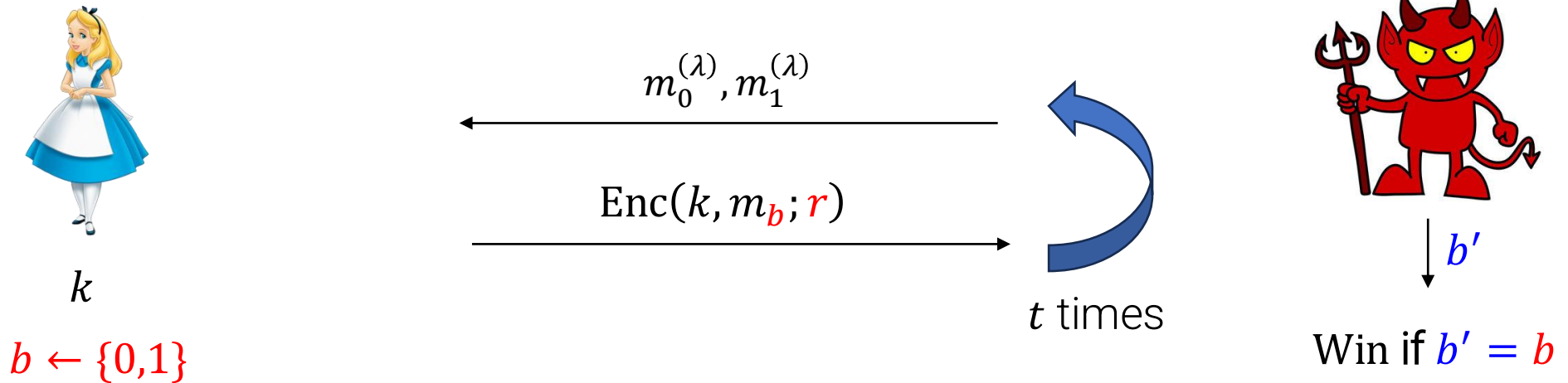
→ Alt. Definition is a special case of Definition

← Hybrid argument & security reduction!

IND-CPA Security

Lemma: Definition and Alt. Definition are equivalent

Proof: ← Hybrid argument & security reduction!



Want to show: $\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$

IND-CPA Security

Lemma: Definition and Alt. Definition are equivalent

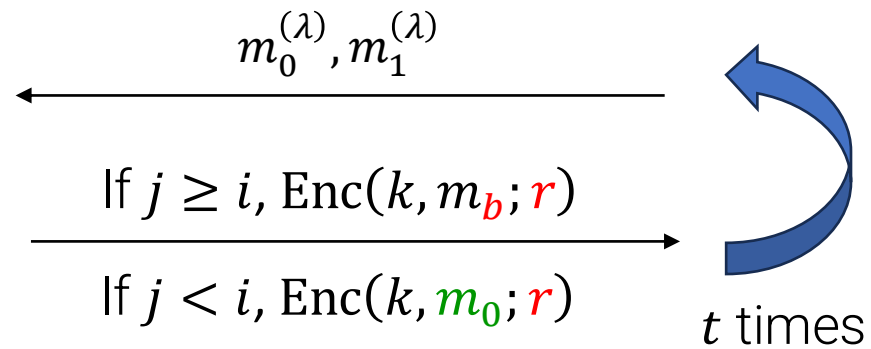
Proof: ← Hybrid argument & security reduction!

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

Win if $b' = b$

Want to show: $\Pr_0[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$

IND-CPA Security

Lemma: Definition and Alt. Definition are equivalent

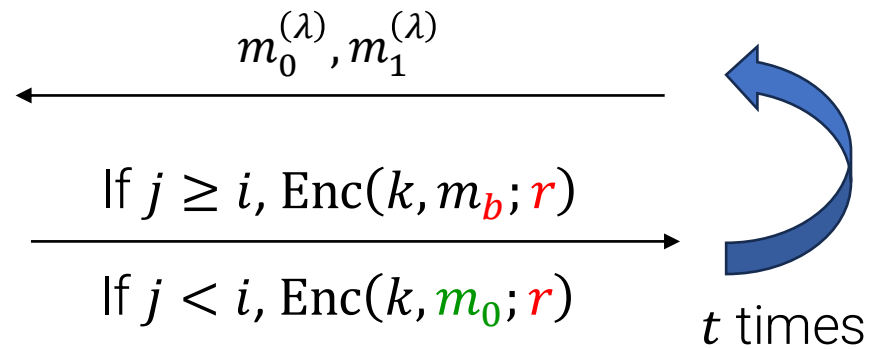
Proof: ← Hybrid argument & security reduction!

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

Win if $b' = b$

Observe: $\Pr_{t+1}[\text{Win}] = \frac{1}{2}$

IND-CPA Security

Lemma: Definition and Alt. Definition are equivalent

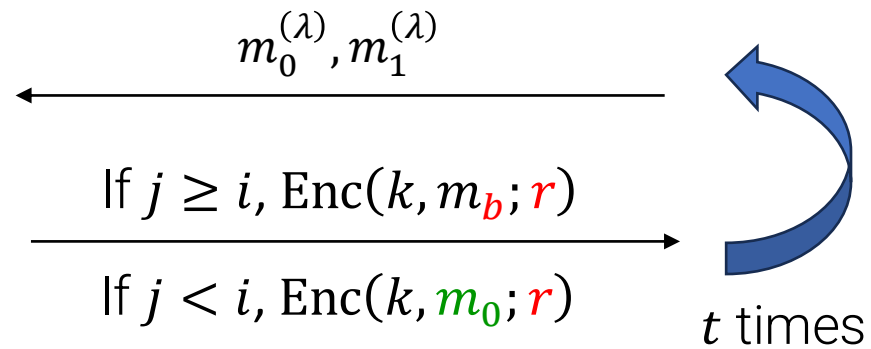
Proof: ← Hybrid argument & security reduction!

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

Win if $b' = b$

$$\text{Claim: } \left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| = \text{negl}(\lambda)$$

IND-CPA Security

$$\text{Claim: } \left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| = \text{negl}(\lambda)$$

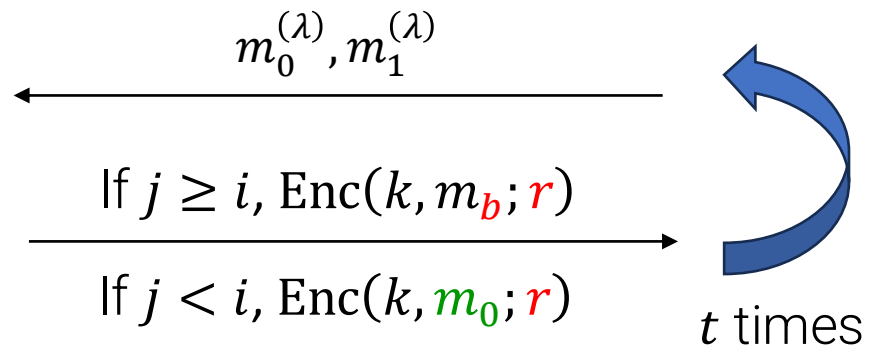
Proof by contradiction: suppose  exists such that $\left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| > \epsilon(\lambda)$

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

Win if $b' = b$

IND-CPA Security

$$\text{Claim: } \left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| = \text{negl}(\lambda)$$

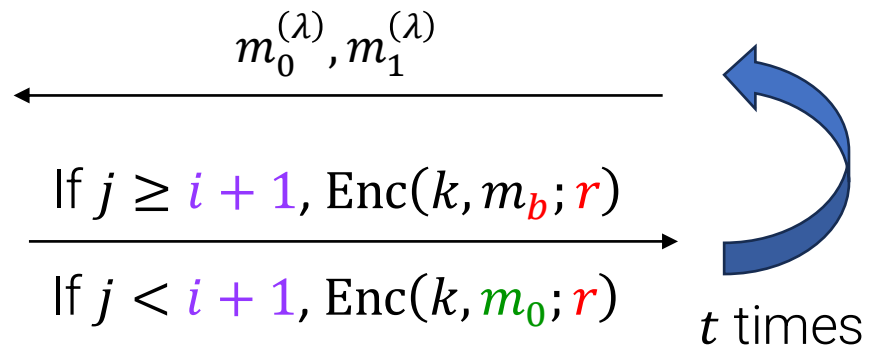
Proof by contradiction: suppose  exists such that $\left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| > \epsilon(\lambda)$

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

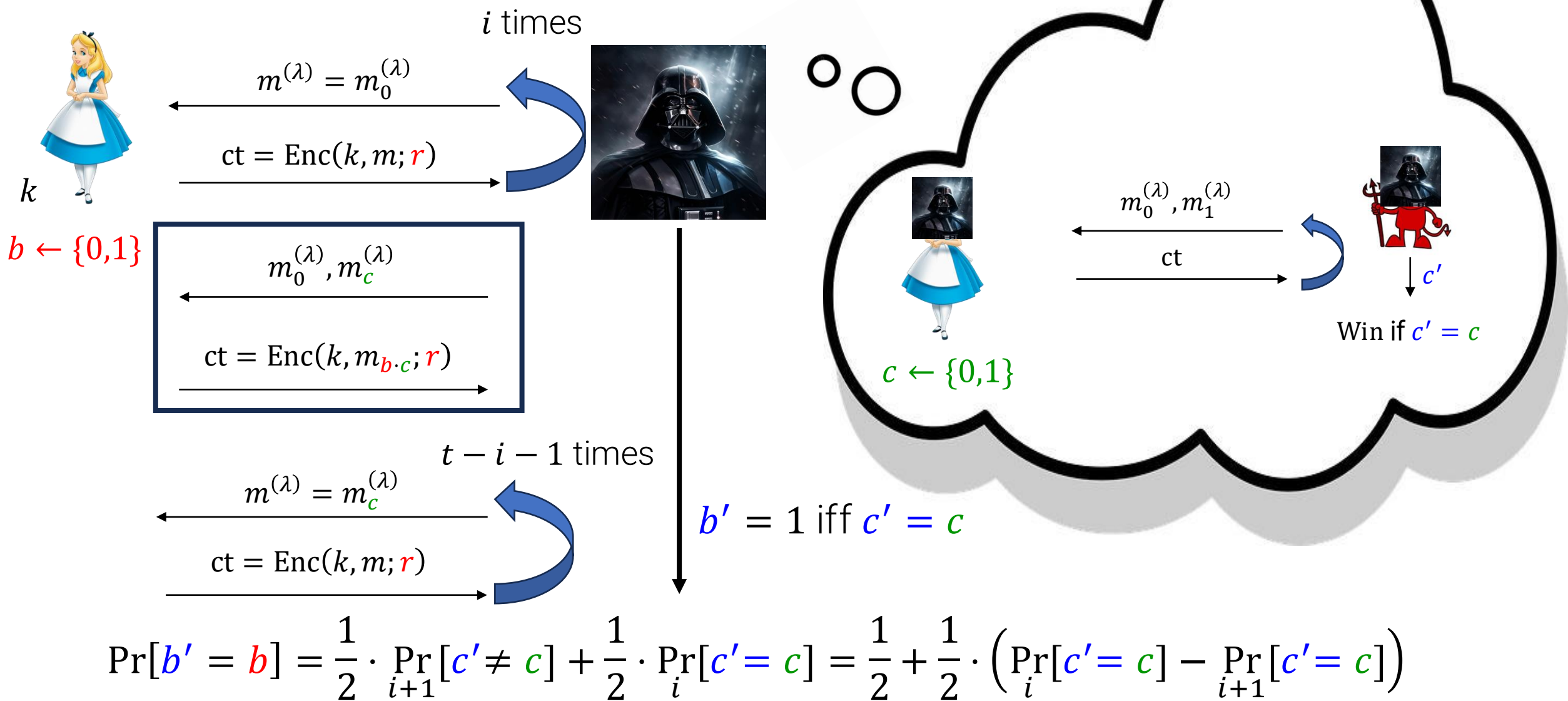
Win if $b' = b$

We will construct



violating the alt. definition!

IND-CPA Security



IND-CPA Security

$$\text{Claim: } \left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| = \text{negl}(\lambda)$$

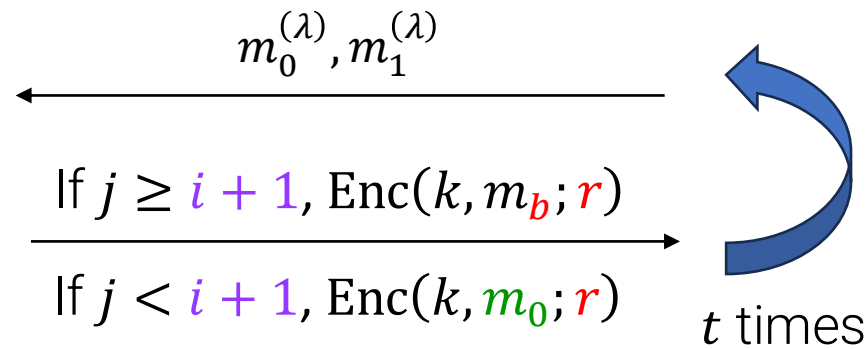
Proof by contradiction: suppose  exists such that $\left| \Pr_{i+1}[\text{Win}] - \Pr_i[\text{Win}] \right| > \epsilon(\lambda)$

Game_{*i*}



k

$b \leftarrow \{0,1\}$



b'

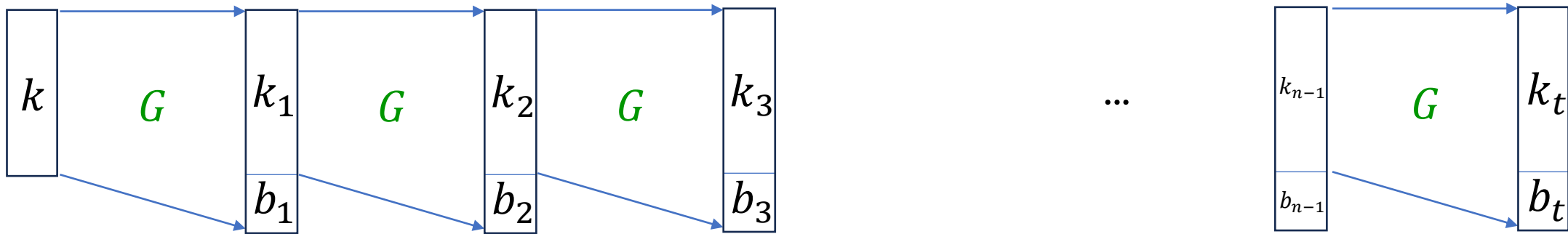
Win if $b' = b$



violates the alt. definition!

Building CPA-secure Encryption

Partial Solution: Stream Cipher



Idea: To encrypt the i^{th} message, use b_i as a one-time pad (PRG length extension)

Secure! (won't prove)

But: parties must keep a counter for how many messages sent so far (and remember k_t).

“Stateful encryption” has many drawbacks.

Building CPA-secure Encryption



k, m

$$ct = G(k) \oplus m$$



k

$$m' = G(k) \oplus ct$$

$$G: \{0,1\}^\lambda \rightarrow \{0,1\}^n$$

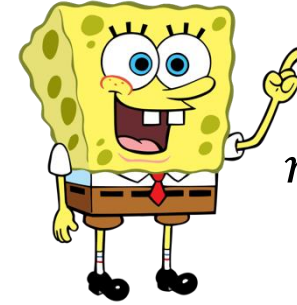
- We need to make use of **randomness**
- Problem: Bob doesn't get access to Alice's random coins... how to decrypt?
- **Idea:** can Alice make the coins **public**?

Building CPA-secure Encryption



k, m, r

$$ct = (r, F(k, r) \oplus m)$$



k

$$m' = F(k, ct_1) \oplus ct_2$$

$$F: \{0,1\}^\lambda \times \{0,1\}^\lambda \rightarrow \{0,1\}^n$$

What property of F would guarantee CPA-security?

Pseudorandom Functions (PRFs)

Challenger



$b \leftarrow \{0,1\}$

$b = 0: k \leftarrow \{0,1\}^\lambda$

$b = 1: F \leftarrow \text{Fun}(\{0,1\}^n \rightarrow \{0,1\}^m)$

$x \in \{0,1\}^n$

$b = 0: \text{PRF}(k, x)$

$b = 1: F(x)$

Adversary



b'

Win if $b' = b$

Definition: An efficiently computable function family $\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

For all polynomial-time

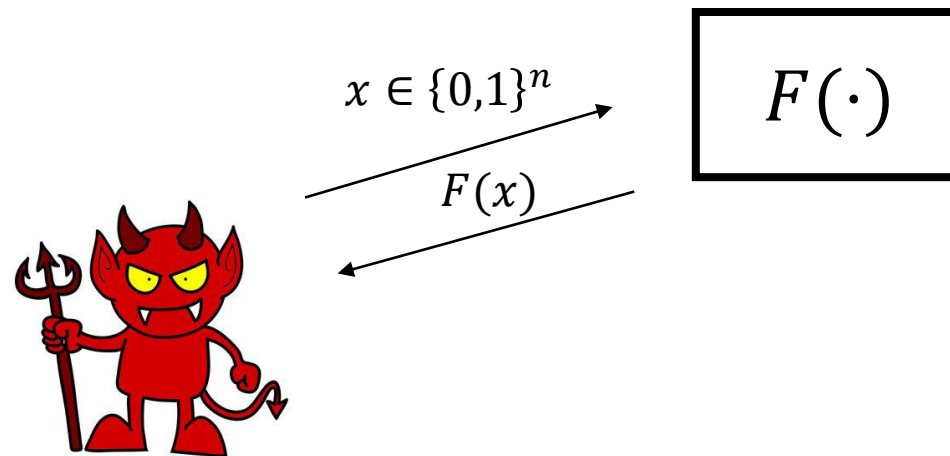


$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Pseudorandom Functions (PRFs)

Another view: “Oracle algorithms”

A poly-time **oracle algorithm** $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\text{poly}(\lambda)$ **at unit cost**



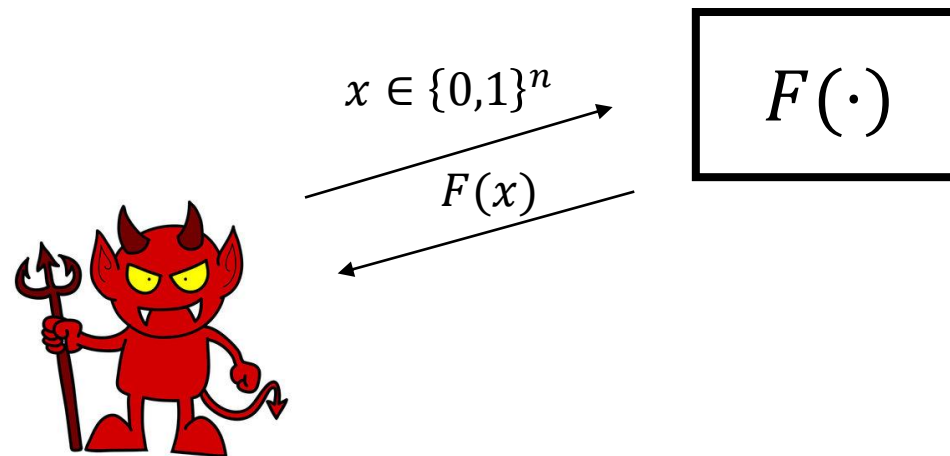
Definition: An efficiently computable function family $\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

For all polynomial-time $A^{F(\cdot)}$, $A^{\text{PRF}(\textcolor{green}{k}, \cdot)}(1^\lambda) \approx_c A^{\textcolor{green}{F}(\cdot)}(1^\lambda)$

Pseudorandom Functions (PRFs)

Another view: “Oracle algorithms”

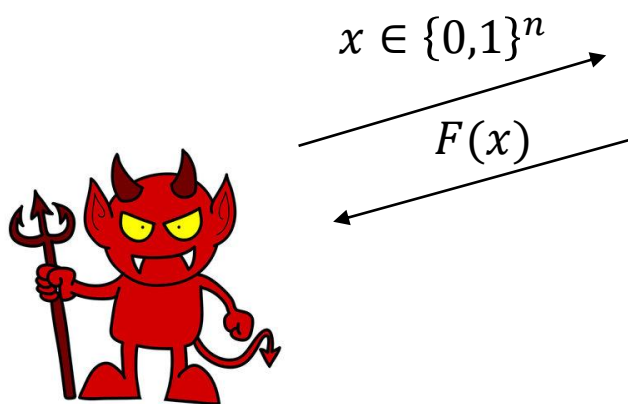
A poly-time **oracle algorithm** $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\text{poly}(\lambda)$ **at unit cost**



Aside: can we efficiently compute $A^{F(\cdot)}(1^\lambda)$? A uniformly random $F(\cdot)$ is likely to be extremely hard to compute.

Observation: we can efficiently evaluate $A^{F(\cdot)}(1^\lambda)$ with a **stateful algorithm**

Pseudorandom Functions (PRFs)



$F(\cdot)$:

- Maintain a list of (x, r_x)
- If query is on the list, output r_x
- If not, sample r_x and add it to the list

Aside: can we efficiently compute $A^{F(\cdot)}(1^\lambda)$? A uniformly random $F(\cdot)$ is likely to be extremely hard to compute.

Observation: we can efficiently evaluate $A^{F(\cdot)}(1^\lambda)$ with a **stateful algorithm**

Pseudorandom Functions (PRFs)

Challenger



$b \leftarrow \{0,1\}$

$b = 0: k \leftarrow \{0,1\}^\lambda$

$b = 1: F \leftarrow \text{Fun}(\{0,1\}^n \rightarrow \{0,1\}^m)$

$x \in \{0,1\}^n$

$b = 0: \text{PRF}(k, x)$

$b = 1: F(x)$

Adversary



b'

Win if $b' = b$

Definition: An efficiently computable function family $\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

For all polynomial-time



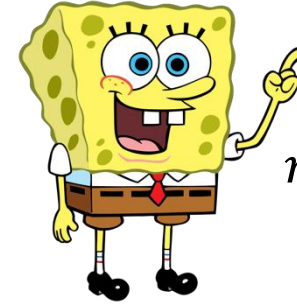
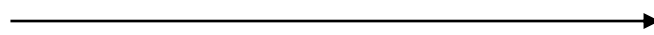
$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Building CPA-secure Encryption



k, m, r

$$ct = (r, \text{PRF}(k, r) \oplus m)$$



k

$$m' = F(k, ct_1) \oplus ct_2$$

$$\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^\lambda \rightarrow \{0,1\}^n$$

Theorem: if PRF is a **PRF family**, then this encryption scheme is **CPA-secure**!

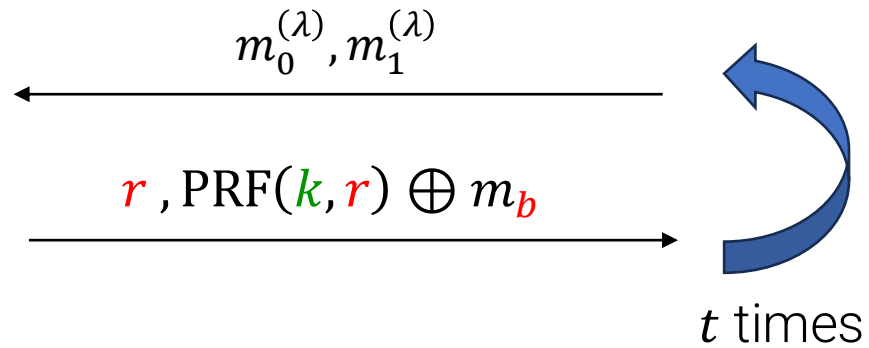
Proof: Hybrid argument & security reduction!

Building CPA-secure Encryption



k

$b \leftarrow \{0,1\}$



b'

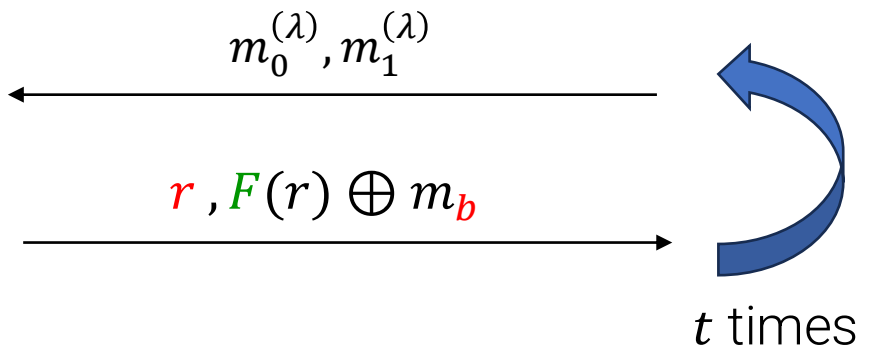
Win if $b' = b$

$\approx_c$
by PRF security
(reduction)



F

$b \leftarrow \{0,1\}$



b'

Win if $b' = b$

Building CPA-secure Encryption



k

$b \leftarrow \{0,1\}$

$\approx \text{negl}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r, s

(need $|r| = \lambda$ so that the t samples of r are distinct!)

t times



b'

Win if $b' = b$



F

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$r, F(r) \oplus m_b$

t times



b'

Win if $b' = b$

Next time: more on PRFs